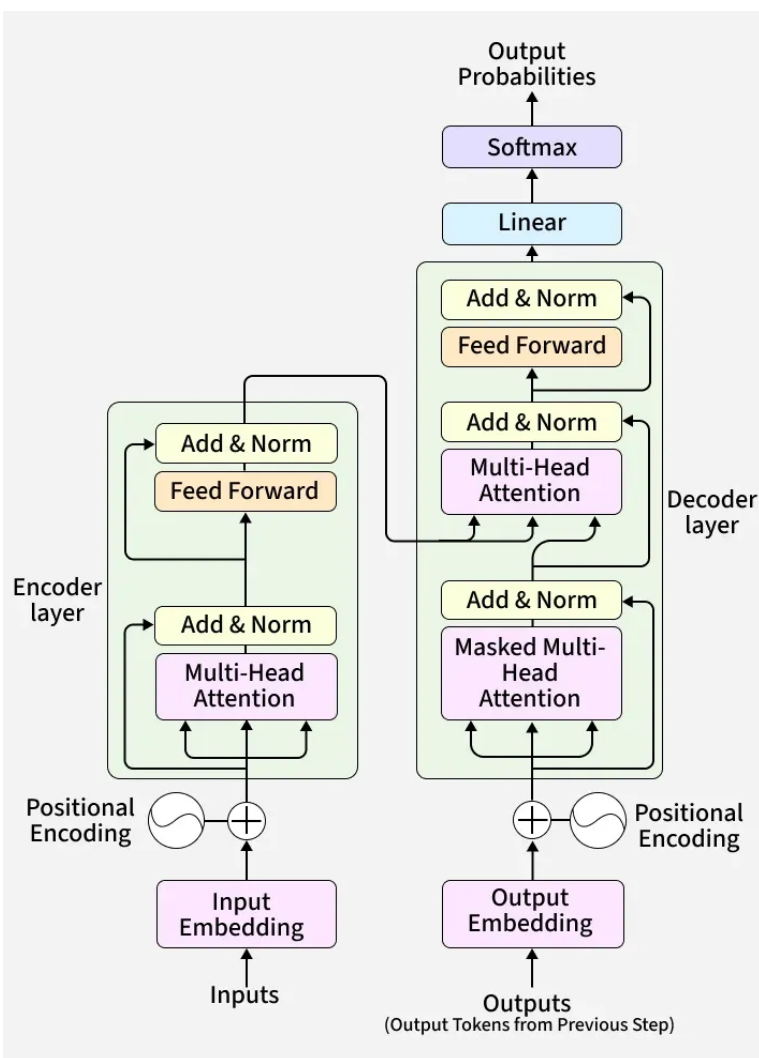


Original Transformer: architecture overview

The original Transformer has **an encoder and a decoder**. Read this diagram from bottom to top: the encoder processes the input, and the decoder uses it to generate the output one token at a time.



- 1. Embeddings + position:** Convert tokens into numerical vectors and add information about their order.
- 2. Encoder:** Self-attention connects input tokens; the feed-forward network processes each token representation.
- 3. Decoder:** Masked attention uses only available output tokens. The middle attention layer is **cross-attention**: it reads the encoder output.
- 4. Add & Norm:** Residual connections carry information forward; layer normalization helps stabilize processing.
- 5. Linear + Softmax:** Produce probabilities for the next output token.

Note: The image shows one encoder layer and one decoder layer. The original 2017 Transformer stacks 6 of each. During generation, the decoder receives tokens produced so far. The following pages explain the architecture families and their model use cases.

1. Encoder-only models

Transformer architectures: a simple guide

Transformer is the architecture family. BERT is a model in that family. The three main architectures are encoder-only, decoder-only and encoder-decoder.

Meaning: reads and understands. An encoder converts input words into numerical representations that capture meaning and context. It can look at words before and after each word.

Example: “The bank approved my loan.” The surrounding words help it understand that bank means a financial institution.

Workflow: Customer review → Encoder → Positive or negative classification. A classification layer turns the encoder representation into a label.

Used for: Text classification, sentiment analysis, identifying names and extracting answers from passages.

Model name / family	Typical use cases
BERT	Sentiment analysis, text classification, identifying names and entities, extracting answers from passages
RoBERTa	Sentiment analysis, text classification, checking relationships between sentences
DistilBERT	Faster text classification and entity extraction with a smaller model
ALBERT	Text classification and question answering with fewer parameters
DeBERTa	Text classification, extractive question answering, checking whether one statement supports another
ELECTRA	Text classification, entity recognition, extractive question answering

2. Decoder-only models

Meaning: generates text step by step. A decoder-only model reads a prompt and generates a response one token at a time. A token can be a word, part of a word or punctuation.

When predicting the next token, it uses tokens already available. It cannot see future tokens. It can also understand the prompt; generation is its typical output behavior.

Example: Prompt: “Explain a bank loan.”
Generation: “A” → “bank” → “loan” → “is” → ...

Used for: Chatbots, writing, coding and summarization.

Model name / family	Typical use cases
GPT	Chatbots, writing, summarization, code generation
Llama	Chatbots, question answering, content generation, RAG applications
Mistral / Mixtral	Chatbots, summarization, coding, RAG applications
Qwen language models	Multilingual conversations, text generation, reasoning, coding
Gemma language models	Text generation, summarization, question answering
Phi language models	Small-model assistants, coding, reasoning tasks

Gemini and Claude: These proprietary model families support generative tasks. Full architecture details are not consistently public for every version, so the families are listed separately without a definitive decoder-only label. Anthropic is the company; Claude is its model family.

Model name / family	Typical use cases
Gemini - Google	Conversations, coding, reasoning, understanding text, images, audio and video; capabilities vary by version
Claude - Anthropic	Writing, document analysis, summarization, coding, reasoning and image understanding; capabilities vary by version

3. Encoder-decoder models

Meaning: reads first, then generates. This architecture has two connected parts.

Encoder: Reads the input and builds numerical representations of its meaning.

Decoder: Uses those representations and the output generated so far to produce the answer one token at a time.

Example: translation

“How are you?” → Encoder represents the input → Decoder generates a translation in another language.

Used for: Translation, summarization, rewriting and other input-to-output tasks.

Model name / family	Typical use cases
T5	Translation, summarization, question answering, rewriting
FLAN-T5	Following instructions, question answering, summarization
BART	Summarization, rewriting, text generation from an input
mT5	Multilingual text generation, translation, summarization
Pegasus	Summarizing articles and documents
MarianMT	Translation between languages
Whisper	Speech-to-text transcription and speech translation into English

Simple way to remember

Model name / family	Typical use cases
Encoder-only	Read input and build an understanding of it.
Decoder-only	Use a prompt to generate text step by step.
Encoder-decoder	Encode the input, then generate output using it.

Important: These are architectural differences, not strict task boundaries. A decoder-only model can also translate or classify text. BERT and its relatives are often called pretrained language models; Whisper is a speech model, rather than a text LLM.